

Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (B.Sc.I.T Semester V)
Dot .Net Core and Progressive Web Application Practical

Practical – V

Roll No.: T008	Name: Aman Manoj Jha
Class: TYIT	Batch: 1
Date of Assignment:21-07-2026	Date/Time of Submission: 21-07-2026

Part B - Program

a) Create a web application to demonstrate the use of different types of Cookies.

Algorithm:

Code:

WebForm1.aspx

```
<%@ Page Language="C#" AutoEventWireup="true" CodeBehind="WebForm1.aspx.cs"
Inherits="Practical_5c.WebForm1" %>
```

```
<!DOCTYPE html>
```

```
<html xmlns="http://www.w3.org/1999/xhtml">
```

```
<head runat="server">
```

```
<title></title>
```

```
</head>
```

```
<form id="form1" runat="server">
```

```
<asp:Label ID="Label1" runat="server" Text="Label"></asp:Label>
```

```
<br />
```

```
<br />
```

```
<asp:Label ID="Label2" runat="server" Text="Label"></asp:Label>
```

```
<br />
```

```
<br />
```

```
<asp:Button ID="Button1" runat="server" OnClick="Button1_Click" Text="View
State" />
```

```
<br />
```

```
<br />
```

```
<asp:TextBox ID="TextBox1" runat="server"></asp:TextBox>
```

```
<br />
```

```
<br />
```

```
<asp:Button ID="Button2" runat="server" OnClick="Button2_Click" Text="Cookie" />
```

```
</form>
```

```
</body>
```

```
</html>
```

WebForm1.aspx.cs

```
using System;
using System.Web;
using System.Web.UI;
using System.Web.UI.WebControls;

namespace Practical_5a
{
    public partial class WebForm1 : System.Web.UI.Page
    {
        protected void Page_Load(object sender, EventArgs e)
        {
            if (IsPostBack)
            {
                int ViewStateVal = Convert.ToInt32(ViewState["count"]) + 1;
                ViewState["count"] = ViewStateVal;
                Label2.Text = "View State = " + ViewStateVal.ToString();
            }
            else
            {
                ViewState["count"] = 1;
            }
        }

        protected void Button1_Click(object sender, EventArgs e)
        {
            Label1.Text = ViewState["count"].ToString();
        }

        protected void Button2_Click(object sender, EventArgs e)
        {
            HttpCookie h = new HttpCookie("name");
            h.Value = TextBox1.Text;

            Response.Cookies.Add(h);
            Response.Redirect("WebForm2.aspx");
        }
    }
}
```

WebForm2.aspx

```
<%@ Page Language="C#" AutoEventWireup="true" CodeBehind="WebForm2.aspx.cs"
Inherits="Practical_5c.WebForm2" %>
```

```
<!DOCTYPE html>
```

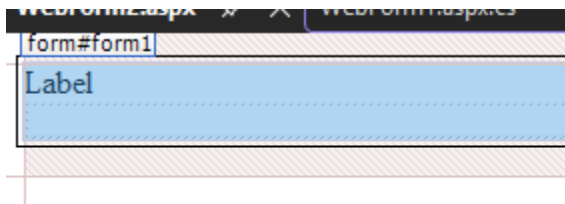
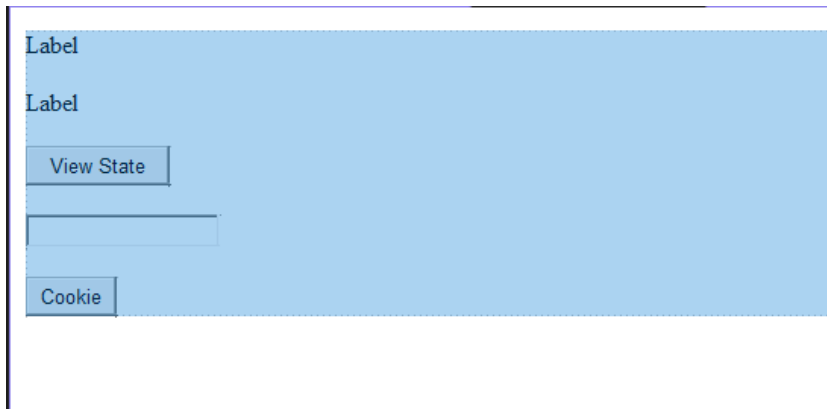
```
<html xmlns="http://www.w3.org/1999/xhtml">
<head runat="server">
  <title></title>
</head>
<body>
  <form id="form1" runat="server">
    <asp:Label ID="Label1" runat="server" Text="Label"></asp:Label>
    <div>
    </div>
  </form>
</body>
</html>
```

WebForm2.aspx.cs

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Web;
using System.Web.UI;
using System.Web.UI.WebControls;

namespace Practical_5c
{
    public partial class WebForm2 : System.Web.UI.Page
    {
        protected void Page_Load(object sender, EventArgs e)
        {
            if (Request.Cookies["name"] != null)
            {
                Response.Write("Welcome"+Request.Cookies["name"].Value);
            }
        }
    }
}
```

Output:



2

View State = 2

View State

Aman

Cookie

WelcomeAman Label

b) Create a Product Table in SQL Server and perform Insert Operation using Entity Framework Core.

Algorithm:

Code:

Program.cs

```
using Microsoft.EntityFrameworkCore;
```

```
var builder = WebApplication.CreateBuilder(args);
```

```
// Add services to the container
```

```
builder.Services.AddControllersWithViews();
```

```

builder.Services.AddDbContext<AppDbContext>(options =>
    options.UseSqlServer(
        builder.Configuration.GetConnectionString("DefaultConnection")
    )
);

var app = builder.Build();

// Configure the HTTP request pipeline
app.UseStaticFiles();

app.UseRouting();

app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}"
);

app.Run();

```

Product.cs

```

public class Product
{
    public int ID { get; set; }
    public string Name { get; set; }
    public int Price { get; set; }
}

```

AppDbContext.cs

```

using Microsoft.EntityFrameworkCore;

public class AppDbContext : DbContext
{
    public AppDbContext(DbContextOptions<AppDbContext> options)
        : base(options)
    {
    }

    public DbSet<Product> Products { get; set; }
}

```

HomeController.cs

```
using Microsoft.AspNetCore.Mvc;

public class HomeController : Controller
{
    private readonly AppDbContext _context;

    public HomeController(AppDbContext context)
    {
        _context = context;
    }

    public IActionResult Index()
    {
        Product p = new Product();

        p.Name = "Laptop";
        p.Price = 50000;

        _context.Products.Add(p);
        _context.SaveChanges();

        return Content("Product Inserted Successfully");
    }
}
```

appsettings.json

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*",
  "ConnectionStrings": {
    "DefaultConnection":
"Server=(localdb)\\MSSQLLocalDB;Database=ProductDB;Trusted_Connection=True;Trust
ServerCertificate=True;"
  }
}
```

```
}
```

InitialCreate.cs

```
using Microsoft.EntityFrameworkCore.Migrations;
```

```
#nullable disable
```

```
namespace Entity_Framework_Core.Migrations
```

```
{
```

```
    /// <inheritdoc />
```

```
    public partial class Initialcreate1 : Migration
```

```
    {
```

```
        /// <inheritdoc />
```

```
        protected override void Up(MigrationBuilder migrationBuilder)
```

```
        {
```

```
            migrationBuilder.CreateTable(
```

```
                name: "Products",
```

```
                columns: table => new
```

```
                {
```

```
                    ID = table.Column<int>(type: "int", nullable: false)
```

```
                        .Annotation("SqlServer:Identity", "1, 1"),
```

```
                    Name = table.Column<string>(type: "nvarchar(max)", nullable: false),
```

```
                    Price = table.Column<int>(type: "int", nullable: false)
```

```
                },
```

```
                constraints: table =>
```

```
                {
```

```
                    table.PrimaryKey("PK_Products", x => x.ID);
```

```
                });
```

```
            }
```

```
        /// <inheritdoc />
```

```
        protected override void Down(MigrationBuilder migrationBuilder)
```

```
        {
```

```
            migrationBuilder.DropTable(
```

```
                name: "Products");
```

```
        }
```

```
    }
```

```
}
```

SQLQuery.sql

```
select * from Products
```

Output:

Product Inserted Successfully

	Id	Name	Price
1	1	Laptop	50000
2	2	Laptop	50000

c) Develop a web application to perform CRUD Operations (Create, Read, Update, Delete) on Employee table using EF Core.

Algorithm:

Code:

EmplooyeCurd

```
<Project Sdk="Microsoft.NET.Sdk.Web">
```

```
<PropertyGroup>
```

```
<TargetFramework>net10.0</TargetFramework>
```

```
<Nullable>enable</Nullable>
```

```
<ImplicitUsings>enable</ImplicitUsings>
```

```
</PropertyGroup>
```

```
<ItemGroup>
```

```
<NuGetAuditSuppress Include="https://github.com/advisories/GHSA-g4vj-cjjj-v7hg" />
```

```
<PackageReference Include="Microsoft.EntityFrameworkCore" Version="10.0.10" />
```

```
<PackageReference Include="Microsoft.EntityFrameworkCore.Design"
Version="10.0.10">
```

```
<PrivateAssets>all</PrivateAssets>
```

```
<IncludeAssets>runtime; build; native; contentfiles; analyzers;
buildtransitive</IncludeAssets>
```

```
</PackageReference>
```

```
<PackageReference Include="Microsoft.EntityFrameworkCore.SqlServer"
Version="10.0.10" />
```



```

    <PackageReference Include="Microsoft.EntityFrameworkCore.Tools"
Version="10.0.10">
    <PrivateAssets>all</PrivateAssets>
    <IncludeAssets>runtime; build; native; contentfiles; analyzers;
buildtransitive</IncludeAssets>
    </PackageReference>
    <PackageReference Include="Microsoft.VisualStudio.Web.CodeGeneration.Design"
Version="10.0.2" />
</ItemGroup>

<ItemGroup>
    <Folder Include="Views\Employee\" />
</ItemGroup>

</Project>

```

Employee.cs

```

namespace EmployeeCRUD.Models
{
    public class Employee
    {
        public int ID { get; set; }
        public string Name { get; set; }
        public string Department { get; set; }
        public double Salary { get; set; }

    }
}

```

AppDbContext

```

using EmployeeCRUD.Models;
using Microsoft.EntityFrameworkCore;
namespace EmployeeCRUD.Data
{
    public class AppDbContext: DbContext
    {
        public AppDbContext(DbContextOptions<AppDbContext> options) : base(options) { }
        public DbSet<Employee> Employees { get; set; }
    }
}

```

appsettings.json

```

{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*",
  "ConnectionStrings": {
    "MyConnection":
"Server=(localdb)\\ProjectModels;Database=EmployeeDB;Trusted_Connection=True;Trusts
erverCertificate=True",
    "AppDbContext":
"Server=(localdb)\\mssqllocaldb;Database=AppDbContext;Trusted_Connection=True;Multi
pleActiveResultSets=true"
  }
}

```

Program.cs

```

using Microsoft.EntityFrameworkCore;
using EmployeeCRUD.Data;

var builder = WebApplication.CreateBuilder(args);

// Add services to the container.
builder.Services.AddControllersWithViews();

builder.Services.AddDbContext<AppDbContext>(options =>
options.UseSqlServer(builder.Configuration.GetConnectionString("MyConnection")));

var app = builder.Build();

// Configure the HTTP request pipeline.
if (!app.Environment.IsDevelopment())
{
    app.UseExceptionHandler("/Home/Error");
    // The default HSTS value is 30 days. You may want to change this for production
    scenarios, see https://aka.ms/aspnetcore-hsts.
    app.UseHsts();
}

app.UseHttpsRedirection();

```

```
app.UseRouting();

app.UseAuthorization();

app.MapStaticAssets();

app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}")
    .WithStaticAssets();

app.Run();
```

EmployeesController.cs

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using EmployeeCRUD.Models;
using EmployeeCRUD.Data;

public class EmployeesController : Controller
{
    private readonly AppDbContext _context;

    public EmployeesController(AppDbContext context)
    {
        _context = context;
    }

    // GET: EMPLOYEES
    public async Task<IActionResult> Index()
    {
        return View(await _context.Employees.ToListAsync());
    }

    // GET: EMPLOYEES/Details/5
    public async Task<IActionResult> Details(int? id)
    {
        if (id == null)
        {
            return NotFound();
        }
    }
}
```

```

        var employee = await _context.Employees
            .FirstOrDefaultAsync(m => m.ID == id);
        if (employee == null)
        {
            return NotFound();
        }

        return View(employee);
    }

    // GET: EMPLOYEES/Create
    public IActionResult Create()
    {
        return View();
    }

    // POST: EMPLOYEES/Create
    // To protect from overposting attacks, enable the specific properties you want to bind to.
    // For more details, see http://go.microsoft.com/fwlink/?LinkId=317598.
    [HttpPost]
    [ValidateAntiForgeryToken]
    public async Task<IActionResult> Create([Bind("ID,Name,Department,Salary")]
Employee employee)
    {
        if (ModelState.IsValid)
        {
            _context.Add(employee);
            await _context.SaveChangesAsync();
            return RedirectToAction(nameof(Index));
        }
        return View(employee);
    }

    // GET: EMPLOYEES/Edit/5
    public async Task<IActionResult> Edit(int? id)
    {
        if (id == null)
        {
            return NotFound();
        }

        var employee = await _context.Employees.FindAsync(id);

```

```

        if (employee == null)
        {
            return NotFound();
        }
        return View(employee);
    }

    // POST: EMPLOYEES/Edit/5
    // To protect from overposting attacks, enable the specific properties you want to bind to.
    // For more details, see http://go.microsoft.com/fwlink/?LinkId=317598.
    [HttpPost]
    [ValidateAntiForgeryToken]
    public async Task<IActionResult> Edit(int? id, [Bind("ID,Name,Department,Salary")]
Employee employee)
    {
        if (id != employee.ID)
        {
            return NotFound();
        }

        if (ModelState.IsValid)
        {
            try
            {
                _context.Update(employee);
                await _context.SaveChangesAsync();
            }
            catch (DbUpdateConcurrencyException)
            {
                if (!EmployeeExists(employee.ID))
                {
                    return NotFound();
                }
                else
                {
                    throw;
                }
            }
            return RedirectToAction(nameof(Index));
        }
        return View(employee);
    }

```

```

// GET: EMPLOYEES/Delete/5
public async Task<IActionResult> Delete(int? id)
{
    if (id == null)
    {
        return NotFound();
    }

    var employee = await _context.Employees
        .FirstOrDefaultAsync(m => m.ID == id);
    if (employee == null)
    {
        return NotFound();
    }

    return View(employee);
}

// POST: EMPLOYEES/Delete/5
[HttpPost, ActionName("Delete")]
[ValidateAntiForgeryToken]
public async Task<IActionResult> DeleteConfirmed(int? id)
{
    var employee = await _context.Employees.FindAsync(id);
    if (employee != null)
    {
        _context.Employees.Remove(employee);
    }

    await _context.SaveChangesAsync();
    return RedirectToAction(nameof(Index));
}

private bool EmployeeExists(int? id)
{
    return _context.Employees.Any(e => e.ID == id);
}
}

```

InitialCreate.cs

```

using Microsoft.EntityFrameworkCore.Migrations;

```

#nullable disable

```
namespace EmployeeCRUD.Migrations
{
    /// <inheritdoc />
    public partial class InitialCreate : Migration
    {
        /// <inheritdoc />
        protected override void Up(MigrationBuilder migrationBuilder)
        {
            migrationBuilder.CreateTable(
                name: "Employees",
                columns: table => new
                {
                    ID = table.Column<int>(type: "int", nullable: false)
                        .Annotation("SqlServer:Identity", "1, 1"),
                    Name = table.Column<string>(type: "nvarchar(max)", nullable: false),
                    Department = table.Column<string>(type: "nvarchar(max)", nullable: false),
                    Salary = table.Column<double>(type: "float", nullable: false)
                },
                constraints: table =>
                {
                    table.PrimaryKey("PK_Employees", x => x.ID);
                });
        }

        /// <inheritdoc />
        protected override void Down(MigrationBuilder migrationBuilder)
        {
            migrationBuilder.DropTable(
                name: "Employees");
        }
    }
}
```

Output:

Welcome

Learn about [building Web apps with ASP.NET Core](#).

Index

[Create New](#)

Name	Department	Salary	
Aman	IT	50000	Edit Details Delete

Create Employee

Name

Aman

Department

IT

Salary

50000

Create

[Back to List](#)

d) Authentication and Authorization in ASP.NET Core.

Algorithm:

Code:

Program.cs

```
var builder = WebApplication.CreateBuilder(args);
```

```
// Add services to the container.
```

```
builder.Services.AddControllersWithViews();
```

```
var app = builder.Build();
```

```
// Configure the HTTP request pipeline.
```

```
if (!app.Environment.IsDevelopment())
```

```
{
```

```
    app.UseExceptionHandler("/Home/Error");
```

```
    // The default HSTS value is 30 days. You may want to change this for production scenarios, see  
    https://aka.ms/aspnetcore-hsts.
```

```
    app.UseHsts();
```

```
}
```

```
app.UseHttpsRedirection();
app.UseRouting();

app.UseAuthorization();

app.MapStaticAssets();

app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}")
    .WithStaticAssets();

app.Run();
```

Index.cshtml

```
<h2>Login</h2>

<form method="post">

    Username :
    <input type="text" name="username" /><br /><br />

    Password :
    <input type="password" name="password" /><br /><br />

    <input type="submit" value="Login" />

</form>

<p style="color:red">
    @ViewBag.Message
</p>
```

HomeController.cs

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Authentication;
using Microsoft.AspNetCore.Authentication.Cookies;
using System.Security.Claims;

namespace AuthDemo.Controllers
```

```

{
    public class HomeController : Controller
    {
        public IActionResult Index()
        {
            return View();
        }

        [HttpPost]
        public async Task<IActionResult> Index(string username, string password)
        {
            if (username == "Aman" && password == "123")
            {
                var claims = new List<Claim>
                {
                    new Claim(ClaimTypes.Name, username)
                };

                var identity = new ClaimsIdentity(claims,
                    CookieAuthenticationDefaults.AuthenticationScheme);

                await HttpContext.SignInAsync(
                    CookieAuthenticationDefaults.AuthenticationScheme,
                    new ClaimsPrincipal(identity));

                return RedirectToAction("Secure");
            }

            ViewBag.Message = "Invalid Username or Password";
            return View();
        }

        [Authorize]
        public IActionResult Secure()
        {
            return Content("Welcome! Login Successful.");
        }

        public async Task<IActionResult> Logout()
        {
            await HttpContext.SignOutAsync(
                CookieAuthenticationDefaults.AuthenticationScheme);

            return RedirectToAction("Index");
        }
    }
}

```

```
}  
}  
}
```

Output:

Practical_5_a Home Privacy

Login

Username :

Password :

© 2026 - Practical_5_a - [Privacy](#)